Months 10 to 12: Fine-tuning

Quick reference · Senior Data Scientist roadmap · Pretrained weights, warm-up, LoRA and Hugging Face fine-tuning, with honest evaluation.

B26 Fine-tuning pretrained models

B26-L1 Transfer learning and loading pretrained weights

Load only the backbone, use strict=False for the new head, and check missing_keys and unexpected_keys every time.

- **state_dict**: A dictionary from parameter names to tensors. torch.save writes it, torch.load reads it (weights_only=True by default since PyTorch 2.6).
- **load_state_dict(strict=False)**: Loads matching names and returns missing_keys and unexpected_keys. Nothing raises if no name matches, so always check the result.
- **Shape mismatch**: Raises even with strict=False. Drop the old head's keys before loading into a model with a new head.
- **Expected missing keys**: Only the new head. Any other missing key means the backbone did not load.

B26-L2 Learning rates for fine-tuning, warm-up and decay

Warm up from 0, decay to 0, count steps as batches times epochs, and step the scheduler after the optimiser every batch.

- **Warm-up**: Raise the rate from 0 in a straight line over the first steps, so early noisy gradients do not damage pretrained weights.
- **Linear decay**: From the end of warm-up, lower the rate in a straight line to 0 at the last step: (total - step) / (total - warmup).
- **Steps**: Batches per epoch, rounded up, times epochs. Call scheduler.step() after optimizer.step() once per batch.
- **transformers 5.22**: get_linear_schedule_with_warmup(optimizer, num_warmup_steps, num_training_steps); TrainingArguments learning_rate 5e-5, warmup_steps a count or a share below 1.

B26-L3 LoRA, fine-tuning with a low-rank update

LoRA freezes W and trains A and B with B at zero, scales the update by alpha / r, and merges into one matrix after training.

- **LoRA output**: y = x @ W.T + (alpha / r) * x @ A.T @ B.T, with W frozen, A (r, d_in), B (d_out, r).
- **LoRA starting point**: A random, B zero, so the update is zero and training starts from the pretrained model.
- **LoRA size**: r * (d_in + d_out) values per layer: 12,288 instead of 589,824 for 768 by 768 at r = 8.
- **Merging**: W + (alpha / r) * B @ A as a new matrix: one plain layer, no extra cost at prediction.

B26-L4 Fine-tuning a text classifier with Hugging Face

Pass num_labels and the label maps, read the load report, use integer labels, call model.train(), and give LoraConfig a task_type.

- **Loading a classifier (transformers 5.22)**: AutoModelForSequenceClassification.from_pretrained(name, num_labels=..., id2label=..., label2id=...). The new head shows as MISSING in the load report.
- **Labels decide the loss**: Integer IDs: cross-entropy. Floats with num_labels > 1: multi-label sigmoid. num_labels=1: mean squared error.
- **Tokenizer call**: tokenizer(texts, padding=True, truncation=True, max_length=..., return_tensors="pt") gives input_ids and attention_mask.
- **peft LoRA for classification (0.21)**: LoraConfig(task_type=TaskType.SEQ_CLS, target_modules=["query", "value"], ...). Without task_type the new head stays frozen.

B26-L5 Token classification, named entities and subword labels

Tokenise with is_split_into_words=True, align with word_ids(), label the first piece of each word, and give everything else -100.

- **BIO tags**: B- begins an entity, I- continues it, O is outside. "New York" is B-LOC I-LOC.
- **word_ids() (transformers 5.22)**: With is_split_into_words=True, the word index of each token: None for special tokens, repeats for split words.
- **Label alignment**: None gets -100, a word's first token gets its label, later pieces get -100. Test with `is None`, never truthiness.
- **-100**: The default ignore_index of nn.CrossEntropyLoss (PyTorch 2.15). Those positions add nothing to the loss.

B26-L6 Evaluating a fine-tuned model

Compare against a trivial baseline and a simple model, report mean and range over seeds, score the test set once, and use entity-level F1 for NER.

- **Baselines for a fine-tuned model**: The trivial baseline (majority class, or all O), a simple model, and the other fine-tuning strategies, on the same validation set.
- **Seeds**: Run at least three, report mean and range. The best seed reports luck.
- **Entity-level F1**: Spans (type, start, end) must match exactly. Precision over predicted spans, recall over true spans.
- **Test set**: Score it once, at the end. Every extra look turns it into validation data.